CSCI 264 — Computer Structure and Assembly Language

Lab 0

Using Unix and GNU C

Nothing is due for this lab. We will work through it together, start to finish. The purpose is to make sure that by the end of the hour every one of you knows exactly where to write a program, how to compile it, and how to run it. Everything else this semester assumes you can do those three things without thinking about them.

You are not expected to know any C yet.

1. Log in to the lab machine with you Linux account.
2. Run `pwd` to see where you are, then `ls` to list the contents of your home directory. It may well be empty.
3. Try the same command again as `ls -a`, then `ls -l`, then `ls -al`. What do `-a` and `-l` each do? These modifiers are called *command line switches*, or just *switches*.
4. Run `man ls`. "man" is short for *manual* — this is the online documentation, and it is installed on every Unix machine you will ever use. Find two switches for `ls` that we have not tried yet and try them out. Press `q` to quit the manual.
5. Make a directory for this course and move into it:

   ```
   mkdir csci264
   cd csci264
   ```

6. Open a second terminal session, so that you can edit in one and compile in the other without closing either.

Writing, compiling, and running a program

7. In one of your two sessions, create a file called `convert.c` using `nano`:

   ```
   nano convert.c
   ```

   The commands you need are listed along the bottom of the screen. The two that matter right now are `^O` to save ("write out") and `^X` to exit. The `^` means the Control key.

   We are using `nano` because its easy You are welcome to use something else if you already know it.

8. We are going to write a program that reads one integer from the user and prints it back out three times: once in decimal, once in octal (base 8), and once in hexadecimal (base 16). The C standard library will do the conversions for us — all three come from the format string:

```
#include <stdio.h>

int main(void) {
    int y;

    printf("Enter an integer: ");
    scanf("%d", &y);

    printf("decimal:     %d\n", y);
    printf("octal:       %o\n", y);
    printf("hexadecimal: %x\n", y);

    return 0;
}
```

9. Save the file and return to your other session. Compile it and run it:

   ```
   gcc convert.c
   ./a.out
   ```

   If you made a typo, `gcc` will report it in the form `convert.c:12:5: error:` — that is the file, the line number, and the column.

10. By default the compiler names the executable `a.out`, which is unhelpful as soon as you have more than one program. Use the `-o` switch to choose the name yourself:

    ```
    gcc -o convert convert.c
    ./convert
    ```

11. Test `convert` with the input `100`. You should see `144` for octal and `64` for hexadecimal. Try a few more values.

A look ahead

12. Now run the compiler with the `-S` switch:

    ```
    gcc -S convert.c
    ```

    This produces a file called `convert.s`. Open it and look at it.

13. Count the lines in that file:

    ```
    wc -l convert.s
    ```

    How many lines of `.s` did your handful of lines of C turn into?

That `.s` file is **assembly code**, and it is what the compiler actually produced from your program before turning it into an executable. It is the subject of the second half of this course. We will come back to this exact command when we look at how compiling really works, and again when you start reading assembly seriously. For now, just note that it exists and that you can look at it whenever you want.